Click Here join official channel

Operating System

Unit 1 Notes

UNIT I - INTRODUCTION TO OPERATING SYSTEM

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

Click Here join official channel

PART 1

BASICS OF OPERATING SYSTEM

- *Introduction to Operating Systems*
- *Goals Of an Operating System*
- *Objectives and Functions of Operating Systems*
- *Evolution of Operating System*
- *Types of Operating System*
- *OS structures-Layered Approach, Monolithic, Microkernel Operating Systems*
- *Operating System operations*
- *Operating System Services*
- *System calls :Introduction*
- *System call Types*
- *Introduction to Linux OS: Components of Linux system, Basic Shell commands*

Click Here join official channel

➢ Introduction to Operating Systems

An operating system is a program that controls the execution of application programs and acts as an interface between the user of a computer and the computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.

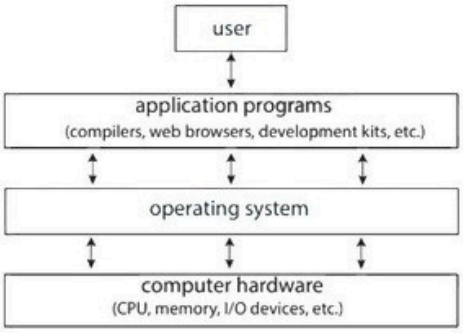


Fig.: An overview of components of computer system.

- **Goals of an Operating System**

  The user goals or requirements should be as follows:

  1. The OS usage should be convenient
  2. Should be easy to use and easy to learn
  3. Should be safe and secure to use and information handling, and security should be robust
  4. Should be fast in response to the user requests

  The system design requirements should be as follows:

  - The OS should be easy to design, maintain and also to implement.
  - Updates should be delivered in an easy way.
  - Adequate security and privacy should be implemented.
  - Resource sharing should be enabled for processes.
  - Should support present software and hardware and should be scalable for future requirements.
  - Should be backward compatible.

- **Objectives of an Operating System**

  - **Efficiency:** The operating system increases the production efficiency. This is because the system configuration takes less time. By default, the operating system handles system tasks such as allocating resources to processes and resolving conflicts between different programs and users. This saves the user time and results in a more efficient result.
  - **Hardware abstraction:** The operating system performs a good job of concealing the computer's intricate details. The user can fully utilise the computer hardware without having to cope with the accompanying difficulties. The operating system coordinates communication between user programs and computer hardware.

Click Here join official channel

- **Convenience:** In the absence of an operating system, users would have to deal with the hardware directly without access to the pre-configured utility packages that come with an operating system. This would make using a computer extremely inconvenient. Operating systems allow users to go right to work on the tasks they want to do without dealing with the burden of setting up the system beforehand.
- **System resource management:** The operating system serves as a neutral arbitrator. It serves a management role in the computer system by ensuring equitable resource distribution among various operations and consumers.

- **Functions of Operating System**
  - **Manages Hardware**

    Controls devices like CPU, memory, keyboard, mouse, printer, etc.

  - **Runs Applications**

    Helps you open and use apps like browsers, games, or word processors.

  - **Manages Files**

    Helps you create, save, open, and delete files and folders.

  - **Multitasking**

    Lets you do many things at once, like listening to music while browsing.

  - **User Interface**

    Gives you a way to interact with the computer (like with icons, windows, and menus).

  - **Security**

    Keeps your data safe with passwords and protects against viruses or unauthorized access.

  - **Memory Management**

    Decides how much memory (RAM) each program gets so everything runs smoothly.

  - **Process Management**

    Controls how programs run and ensures each gets enough CPU time.

Click Here join official channel

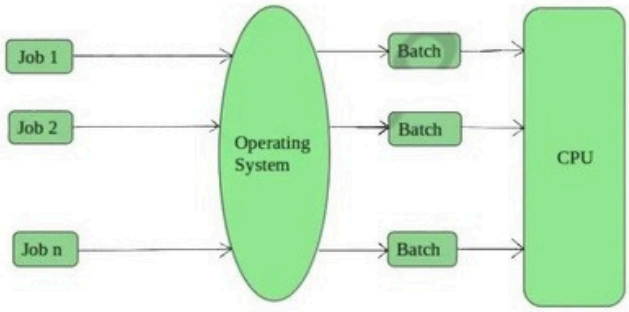
- **Evolution of an Operating System**

Generation	Period	Computer Architecture	Development of an OS
First	1940s – 1950s	Vacuum tubes-based technology, plug boards and punched cards, magnetic core memories	No operating system
Second	1950s – 1960s	Mainframe computers, line printers, magnetic tapes, assemblers, linkers, loaders, compilers, FORTRAN, COBOL	Automated job sequencing Resident monitor Batch systems Offline operation with magnetic tapes
Third	1960s – 1980s	IC-based technology, Minicomputer Magnetic disk	Multi-programming Multi-programmed batch systems Time-sharing multi-user systems UNIX
Fourth	1980s – Present	LSI- and VLSI-based technology, Microcomputer	MS-DOS Apple Macintosh Windows Multi-tasking Multi-threading X Windows Network operating systems Distributed operating systems

Click Here join official channel

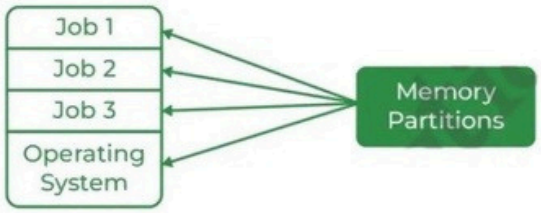
- **Types of an Operating System**

1. **Batch Operating System:** This type of operating system does not interact with the computer directly. There is an operator which takes similar jobs having the same requirement and group them into batches. It is the responsibility of the operator to sort jobs with similar needs.



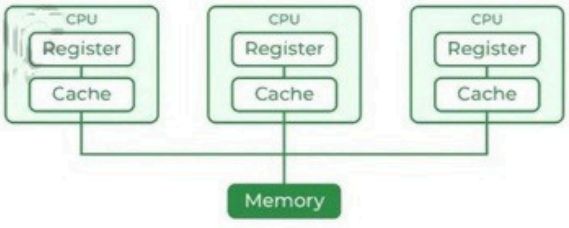
Examples: Payroll System, Bank Statements, etc.

2. **Multiprogramming Operating System:** Multiprogramming is an extension to batch processing where the CPU is always kept busy. Each process needs two types of system time: CPU time and IO time.



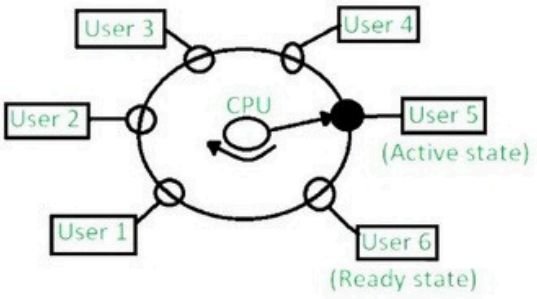
Examples: Windows O/S, UNIX O/S, Microcomputers such as XENIX, MP/M, and ESQview etc.

3. **Multiprocessing Operating System:** In Multiprocessing, Parallel computing is achieved. There are more than one processor present in the system which can execute more than one process at the same time. This will increase the throughput of the system.

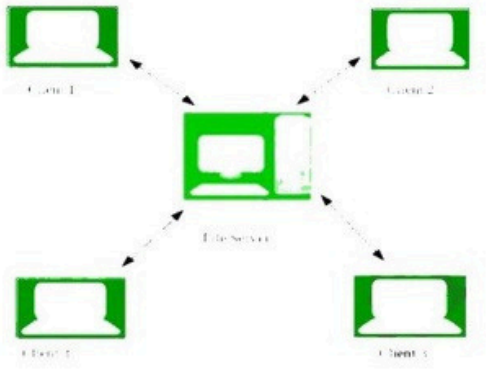


4. **Time-Sharing Operating Systems:** Each task is given some time to execute so that all the tasks work smoothly. Each user gets the time of CPU as they use a single system. These systems are also known as Multitasking Systems. The task can be from a single user or different users also. The time that each task gets to execute is called quantum. After this time interval is over OS switches over to the next task.
Examples: Multics, Unix, etc.

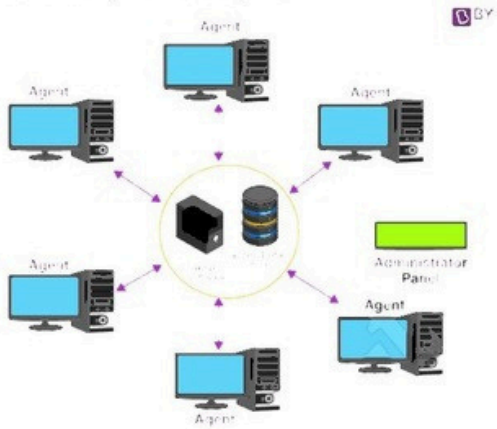
Click Here join official channel



5. **Distributed Operating System:** These are referred to as **loosely coupled systems** or distributed systems. These system's processors differ in size and function. The major benefit of working with these types of the operating system is that it is always possible that one user can access the files or software which are not actually present on his system but some other system connected within this network i.e., remote access is enabled within the devices connected in that network.
Examples: LOCUS, etc.



6. **Network Operating System:** These systems run on a server and provide the capability to manage data, users, groups, security, applications, and other networking functions. These types of operating systems allow shared access of files, printers, security, applications, and other networking functions over a small private network. these computers are popularly known as **tightly coupled systems**.
Example: Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD, etc.



7. **Real-Time Operating System:** These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval

Click Here join official channel

is called response time. Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Examples: Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.



Click Here join official channel

➢ Operating System Structure

1. Simple Structure

There are many operating systems that have a rather simple structure. These started as small systems and rapidly expanded much further than their scope. A common example of this is MS-DOS. It was designed simply for a niche amount for people. The interfaces and levels of functionality are not well separated. MS-DOS is an example of such an operating system. In MS-DOS, application programs are able to access the basic I/O routines. These types of operating systems cause the entire system to crash if one of the user programs fails.

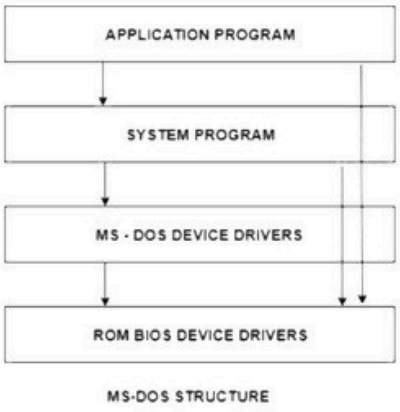


Fig.: Simple Structure

Advantages of Simple Structure

- It delivers better application performance because of the few interfaces between the application program and the hardware.
- It is easy for kernel developers to develop such an operating system.

Disadvantages of Simple Structure

- The structure is very complicated, as no clear boundaries exist between modules.
- It does not enforce data hiding in the operating system.

2. Monolithic OS

The monolithic operating system is a very basic operating system in which file management, memory management, device management, and process management are directly controlled within the kernel. The kernel can access all the resources present in the system. Operating systems that use monolithic architecture were first time used in the 1970s.The monolithic operating system *is* also known as the monolithic kernel The monolithic kernel acts as a virtual machine that controls all hardware parts.

Advanatages of Monolithic Structure

- Performance of Monolithic structure is fast as since everything runs in a single block, therefore communication between components is quick.
- It is easier to build because all parts are in one code block.

Disadvanatages of Monolithic Architecture

- It is hard to maintain as a small error can affect entire system.
- There are also some security risks in the Monolithic architecture.

Click Here join official channel

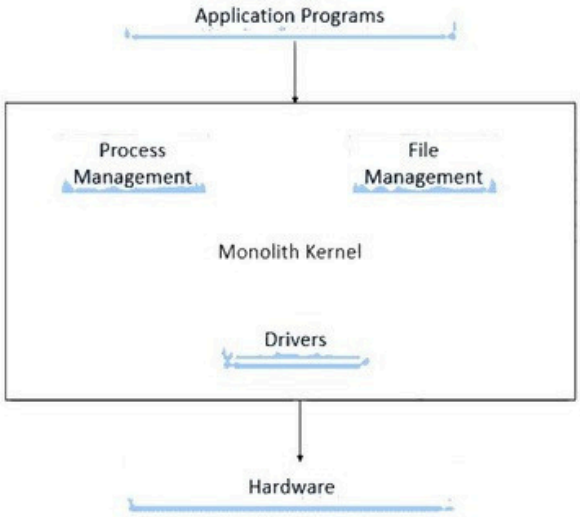


Fig.: Monolithic OS Structure

3. Layered Structure

A system can be made modular in many ways and one of the methods is the layered approach in which the operating system is broken into several layers. Generally, the structure of an operating system depends on how the various common components are interconnected with each other and how it is combined with the kernel. The layered structure of an operating system is the improvised approach of the early monolithic systems.

Advantages :

There are several advantages to this design :

1. Modularity : This design promotes modularity as each layer performs only the tasks it is scheduled to perform.
2. Easy debugging : As the layers are discrete so it is very easy to debug. Suppose an error occurs in the CPU scheduling layer, so the developer can only search that particular layer to debug, unlike the Monolithic system in which all the services are present together.
3. Easy update : A modification made in a particular layer will not affect the other layers.
4. No direct access to hardware : The hardware layer is the innermost layer present in the design. So a user can use the services of hardware but cannot directly modify or access it, unlike the Simple system in which the user had direct access to the hardware.
5. Abstraction : Every layer is concerned with its own functions. So the functions and implementations of the other layers are abstract to it.

Disadvantages :

Though this system has several advantages over the Monolithic and Simple design, there are also some disadvantages as follows.

1. Complex and careful implementation : As a layer can access the services of the layers below it, so the arrangement of the layers must be done carefully. For example, the backing storage layer uses the services of the memory management layer. So it must be kept below the memory management layer. Thus with great modularity comes complex implementation.
2. Slower in execution : If a layer wants to interact with another layer, it sends a request that has to travel through all the layers present in between the two interacting layers. Thus it increases response time, unlike the Monolithic system which is faster than this. Thus an increase in the number of layers may lead to a very inefficient design.

Click Here join official channel

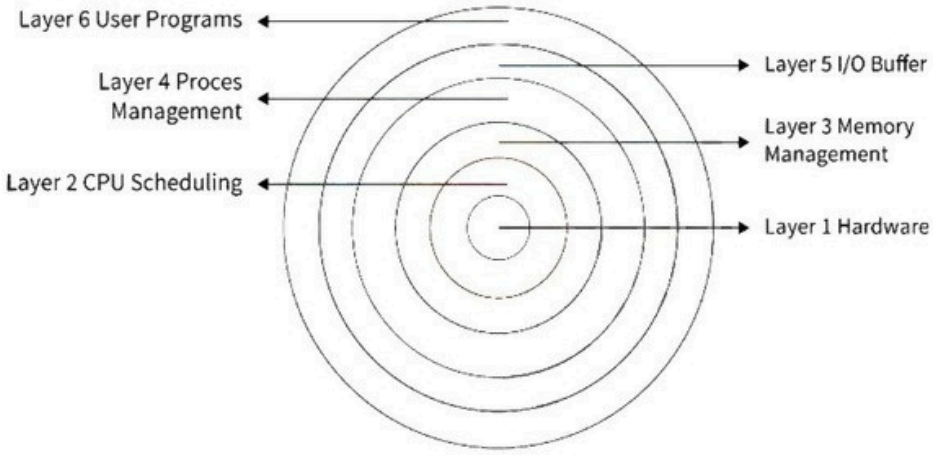


Fig.: Layered Structure

4.Micro Kernel OS

Micro kernel is the type of the operating system architecture which is useful in the same way as the other architectures are and is used for file management, memory management and the scheduling of the processes. The basic idea of microkernel design is to achieve high reliability by splitting the operating system up into small, well-defined modules. The microkernel OS runs in kernel mode.

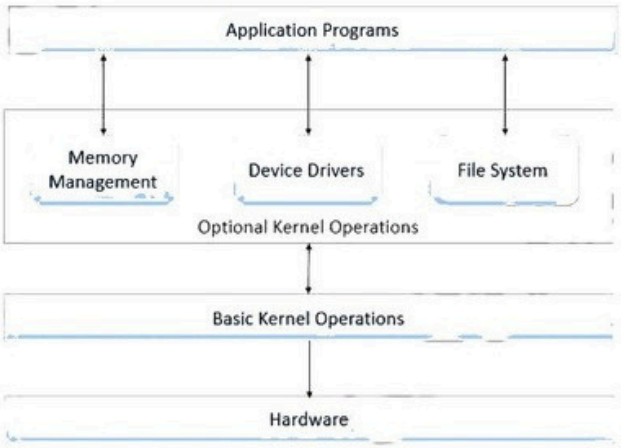
The main function of microkernel is to provide a communication facility between the client program and various services that are running in user space.

Advantages

- **Reliability**: A crash in one service doesn't bring down the whole OS.
- **Security**: Services are isolated.
- **Portability**: Easier to port to different hardware.

Disadvantages

- **Performance penalty**: Due to increased IPC overhead.
- **Complex service management**: More care needed to coordinate services.



Click Here join official channel

4. Client Server OS

Client/Server communication involves two components, namely a client and a server. They are usually multiple clients in communication with a single server. The clients send requests to the server and the server responds to the client requests. In a client-server OS structure, services (file system, printing, etc.) are treated as servers, and applications are clients. They communicate using message passing.

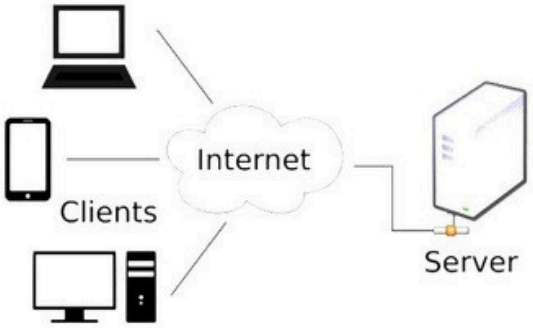


Fig. Client Server OS Structure

Click Here join official channel

➢ Operating System Operations

1. **Multiprogramming and Multitasking**

   One of the most important aspects of operating systems is the ability to run multiple programs, as a single program cannot keep CPU or the I/O devices busy at all times. Users typically want to run more than one program at a time as well. Multiprogramming increases CPU utilization, as well as keeping users satisfied, by organizing programs so that the CPU always has one to execute. In a multi-programmed system, a program in execution is termed a process.

   Multitasking is a logical extension of multiprogramming. In multitasking systems, the CPU executes multiple processes by switching among them, but the switches occur frequently, providing the user with a fast response time.

2. **Dual mode and multi-mode**

   The concept of modes of operation in operating system can be extended beyond the dual mode. This is known as the multimode system. In those cases the more than 1 bit is used by the CPU to set and handle the mode.

   An example of the multimode system can be described by the systems that support virtualisation. These CPU's have a separate mode that specifies when the virtual machine manager (VMM) and the virtualisation management software is in control of the system.

3. **Timer**

   We must ensure that the operating system maintains control over the CPU. We cannot allow a user program to get stuck in an infinite loop or to fail to call system services and never return control to the operating system. To accomplish this goal, we can use a timer. A timer can be set to interrupt the computer after a specified period. The period may be fixed or variable. A variable timer is generally implemented by a fixed-rate clock and a counter. The operating system sets the counter.

Click Here join official channel

➢ Functions of An Operating Systems

1. Memory Management: It is the management of the main or primary memory. Whatever program is executed, it has to be present in the main memory. Main memory is a quick storage area that may be accessed directly by the CPU. When the program is completed, the memory region is released and can be used by other programs. Therefore, there can be more than one program present at a time. Hence, it is required to manage the memory.

The operating system:

- Allocates and deallocates the memory.
- Keeps a record of which part of primary memory is used by whom and how much.
- Distributes the memory while multiprocessing.
- In multiprogramming, the operating system selects which processes acquire memory when and how much memory they get.

2. Processor Management/Scheduling: Every software that runs on a computer, whether in the background or in the frontend, is a process. Processor management is an execution unit in which a program operates. The operating system determines the status of the processor and processes, selects a job and its processor, allocates the processor to the process, and de-allocates the processor after the process is completed.

Certain algorithms used for CPU scheduling are as follows:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Round-Robin Scheduling
- Priority-based scheduling etc.

3. Device Management: An operating system regulates device connection using drivers. The processes may require devices for their use. This management is done by the OS. The OS:

- Allocates and deallocates devices to different processes.
- Keeps records of the devices.
- Decides which process can use which device for how much time.

4. **File Management:** The operating system manages resource allocation and de-allocation. It specifies which process receives the file and for how long. It also keeps track of information, location, uses, status, and so on. These groupings of resources are referred to as file systems. The files on a system are stored in different directories. The OS:

    - Keeps records of the status and locations of files.
    - Allocates and deallocates resources.
    - Decides who gets the resources.

Click Here join official channel

5. **Storage Management:** Storage management is a procedure that allows users to maximize the utilization of storage devices while also protecting data integrity on whatever media on which it lives. Network virtualization, replication, mirroring, security, compression, deduplication, traffic analysis, process automation, storage provisioning, and memory management are some of the features that may be included. The operating system is in charge of storing and accessing files. The creation of files, the creation of directories, the reading and writing of data from files and directories, as well as the copying of the contents of files and directories from one location to another are all included in storage management.

➢ Operating System Services and Interface

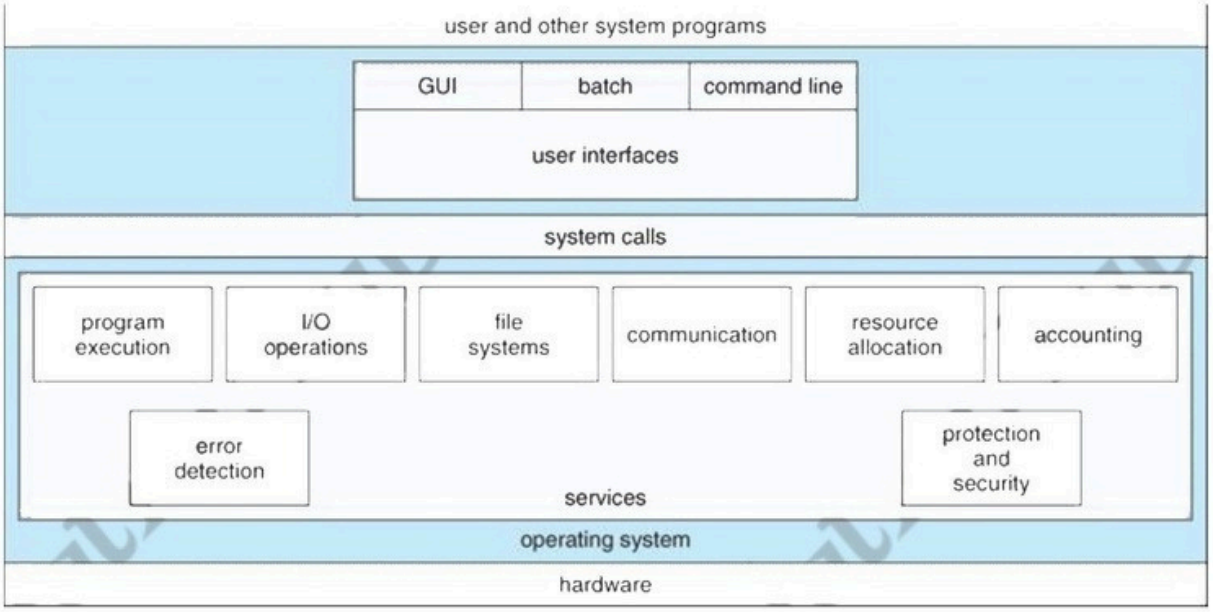


Fig.: A view of an OS Services

1. **User interface**: Almost all operating systems have a user interface (UI). This interface can take several forms. Most commonly, a graphical user interface (GUI) is used. Mobile systems such as phones and tablets provide a touch-screen interface, enabling users to slide their fingers across the screen or press buttons on the screen to select choices. Another option is a command-line interface (CLI), which uses text commands and a method for entering them. Some systems provide two or all three of these variations.

2. **Program execution**: The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).

3. **I/O operations**: A running program may require I/O, which may involve a file or an I/O device. For specific devices, special functions may be desired (such as reading from a network interface or writing to a file system). For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide a means to do I/O.

Click Here join official channel

4. **File-system manipulation**: Programs need to read and write files and directories. They also need to create and delete them by name, search for a given file, and list file information. Some operating systems include permissions management to allow or deny access to files or directories based on file ownership. Many operating systems provide a variety of file systems, sometimes to allow personal choice and sometimes to provide specific features or performance characteristics.

5. **Communications:** There are many circumstances in which one process needs to exchange information with another process. Such communication may occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a network. Communications may be implemented via shared memory, in which two or more processes read and write to a shared section of memory, or message passing, in which packets of information in predefined formats are moved between processes by the operating system.

6. **Error detection**: The operating system needs to be detecting and correcting errors constantly. Errors may occur in the CPU, memory, in I/O devices and in the user program. For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing. Sometimes, it has no choice but to halt the system. At other times, it might terminate an error-causing process or return an error code to a process for the process to detect and possibly correct.

➢ INTERFACE OF AN OS

1. **Command Line Interpreter:**
   Most operating systems, including Linux, UNIX, and Windows, treat the command interpreter as a special program that is running when a process is initiated or when a user first logs on systems. On systems with multiple command interpreters to choose from, the interpreters are known as shells.
   The main function of the command interpreter is to get and execute the next user-specified command. Many of the commands given at this level manipulate files: create, delete, list, print, copy, execute, and so on. The various shells available on UNIX systems operate in this way.

2. **Graphical User Interface:**
   A second strategy for interfacing with the operating system is through a user-friendly graphical user interface, or GUI. Here, rather than entering commands directly via a command-line interface, users employ a mouse-based window and-menu system characterized by a desktop metaphor. The user moves the mouse to position its pointer on images, or icons, on the screen (the desktop) that represent programs, files, directories, and system functions. The first GUI appeared on the Xerox Alto computer in 1973. However, graphical interfaces became more widespread with the advent of Apple Macintosh computers in the 1980s.

Click Here join official channel

3. **Screen Touch Interface:**
   Because a either a command-line interface or a mouse-and-keyboard system is impractical for most mobile systems, smartphones and handheld tablet computers typically use a touch-screen interface. Here, users interact by making gestures on the touch screen— for example, pressing and swiping fingers across the screen. Although earlier smartphones included a physical keyboard, most smartphones and tablets now simulate a keyboard on the touch screen.

➢ System Calls and its Types

A system call is a function that a user program uses to ask the operating system for a particular service. User programmers can communicate with the operating system to request its services using the interface that is created by a system call.

A system calls in computing is the programmatic method by which a computer programme asks the kernel of the operating system it is running on for a service. Programs can interact with the operating system by making a system call. When a computer programme requests something from the kernel of the operating system, it performs a system call.

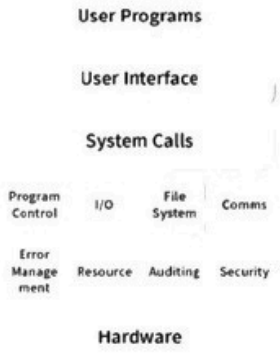


Fig.: Overview of System Call

System call uses Application Programming Interfaces to provide operating system services to user programmes (API). It offers a point of contact between processes and the operating system so that user-level processes can ask for the operating system's assistance. The only ways to access the kernel system are through system calls. System calls are required for all programmes that require resources.

Click Here join official channel

➢ **Types of System Call**

1. Process control
   - create process, terminate process
   - load, execute
   - get process attributes, set process attributes
   - wait event, signal event
   - allocate and free memory
2. File management
   - create file, delete file
   - open, close
   - read, write, reposition
   - get file attributes, set file attributes
3. Device management
   - request device, release device
   - read, write, reposition
   - get device attributes, set device attributes
   - logically attach or detach devices
4. Information maintenance
   - get time or date, set time or date
   - get system data, set system data
   - get process, file, or device attributes
   - set process, file, or device attributes
5. Communications
   - create, delete communication connection
   - send, receive messages
   - transfer status information
   - attach or detach remote devices
6. Protection
   - get file permissions
   - set file permissions

➢ SYSTEM PROGRAMS

1. **File Management:** A file is a collection of specific information stored in the memory of a computer system. File management is defined as the process of manipulating files in the computer system, its management includes the process of creating, modifying and deleting files.

➢ **Status Information:** Information like date, time amount of available memory, or disk space is asked by some users. Others providing detailed performance, logging, and debugging information which is more complex. All this information is formatted

Click Here join official channel

and displayed on output devices or printed. Terminal or other output devices or files or a window of GUI is used for showing the output of programs.

- **File Modification:** For modifying the contents of files we use this. For Files stored on disks or other storage devices, we used different types of editors. For searching contents of files or perform transformations of files we use special commands.

- **Programming-Language support:** For common programming languages, we use Compilers, Assemblers, Debuggers, and interpreters which are already provided to users. It provides all support to users. We can run any programming language. All languages of importance are already provided.

- **Program Loading and Execution:** When the program is ready after Assembling and compilation, it must be loaded into memory for execution. A loader is part of an operating system that is responsible for loading programs and libraries. It is one of the essential stages for starting a program. Loaders, relocatable loaders, linkage editors, and Overlay loaders are provided by the system.

- **Communications:** Virtual connections among processes, users, and computer systems are provided by programs. Users can send messages to another user on their screen, User can send e-mail, browsing on web pages, remote login, the transformation of files from one user to another.

➢ System Boot

Booting is the process of starting a computer. It can be initiated by hardware such as a button press or by a software command. After it is switched on, a CPU has no software in its main memory, so some processes must load software into memory before execution. This may be done by hardware or firmware in the CPU or by a separate processor in the computer system.

Restarting a computer also is called rebooting, which can be "***hard***", e.g., after electrical power to the CPU is switched from off to on, or "***soft***", where the power is not cut. On some systems, a soft boot may optionally clear RAM to zero. Hard and soft booting can be initiated by hardware such as a button press or a software command. Booting is complete when the operative runtime system, typically the operating system and some applications, is attained.

Types of Booting

1. **Cold Booting:** When the computer starts for the first time or is in a shut-down state and switch on the power button to start the system, this type of process to start the computer is called cold booting. During cold booting, the system will read all the instructions from the

Click Here join official channel

ROM (BIOS) and the Operating System will be automatically get loaded into the system. This booting takes more time than Hot or Warm Booting.

2. **Warm Booting:** Warm or Hot Booting process is when computer systems come to no response or hang state, and then the system is allowed to restart during on condition. It is also referred to as rebooting. There are many reasons for this state, and the only solution is to reboot the computer. Rebooting may be required when we install new software or hardware. The system requires a reboot to set software or hardware configuration changes, or sometimes systems may behave abnormally or may not respond properly. In such a case, the system has to be a force restart.

Click Here join official channel

Introduction to Linux operating system

Linux is an open-source operating system kernel first released by **Linus Torvalds** in 1991. It is a Unix-like system that provides a stable, secure, and powerful platform for a wide range of computing environments — from personal desktops and servers to embedded devices and supercomputers.

- The term **"Linux"** often refers to the entire OS built around the Linux kernel, including GNU utilities, libraries, and other software.
- Often called **GNU/Linux** to acknowledge the GNU components.
- It is **free software**, licensed under the GNU General Public License (GPL), meaning anyone can use, modify, and redistribute it.

History and Evolution

- **UNIX roots:** Linux is inspired by the UNIX operating system developed in the 1970s at AT&T Bell Labs.
- **Linus Torvalds' announcement:** In 1991, Linus posted the first version of Linux kernel to the public, inviting collaboration.
- **GNU Project:** Started by Richard Stallman in the 1980s to create a free UNIX-like OS. Many GNU tools (like bash shell, gcc compiler) combined with Linux kernel create a fully functional OS.
- Since then, Linux has grown with contributions from thousands worldwide.
- Popular distributions emerged: **Debian, Red Hat, Ubuntu, Fedora, CentOS, Arch Linux, SUSE**, etc.

Features of Linux

a. Open Source

- Source code is freely available.
- Users and developers can study, modify, and share the code.

b. Multiuser and Multitasking

- Supports multiple users logged in simultaneously without interference.
- Allows many programs to run at the same time efficiently.

c. Portability

- Runs on many hardware platforms, from microcontrollers to supercomputers.

d. Security and Stability

- Linux is renowned for its security features and uptime.
- Its permission and ownership model protect against unauthorized access.
- Regular updates and patches improve security.

e. Modular Kernel

- The kernel can load/unload modules dynamically (e.g., device drivers).

Click Here join official channel

f. Rich Development Environment

- Supports many programming languages and development tools.
- Preferred OS for developers and system administrators.

g. Networking

- Powerful networking stack; widely used in servers and networking devices.

Components of the Linux System

Linux is a complex software system composed of several key components that work together to provide a fully functional operating environment. These components can be broadly divided into:

1. Kernel
2. System Libraries
3. System Utilities
4. Shell
5. User Applications

1. Kernel

What is the Kernel?

The **kernel** is the core part of the Linux operating system. It acts as a bridge between the hardware and software applications. The kernel is responsible for managing the system's resources and communicating with the hardware.

Key Responsibilities:

- **Process management:** Creating, scheduling, and terminating processes.
- **Memory management:** Allocating and managing system memory, including virtual memory.
- **Device management:** Interfacing with hardware devices via device drivers.
- **File system management:** Managing files on storage devices.
- **System calls and security:** Provides an interface for user applications to interact safely with hardware.

Types of Kernels:

- Linux uses a **monolithic kernel** with modular capabilities, meaning the kernel runs as a single large process but allows dynamic loading/unloading of modules like drivers.

Kernel Modules:

- Kernel functionality can be extended by loading modules (e.g., device drivers) without rebooting.

Click Here join official channel

2. System Libraries

What are System Libraries?

System libraries are collections of functions or routines used by programs to perform common tasks without needing to interact directly with the kernel.

Purpose:

- Provide a standard interface for programs.
- Abstract the complexity of kernel services.
- Most system libraries are written in C.

Example:

- **GNU C Library (glibc):** The most important system library in Linux. Provides standard C functions like `printf()`, `scanf()`, file operations, memory allocation, etc.

How System Libraries Work:

- Applications call functions from these libraries.
- Libraries internally make system calls to the kernel when needed.

3.System Utilities

What are System Utilities?

System utilities are programs that perform individual, specific functions related to file management, system monitoring, disk management, and more.

Examples of Utilities:

- File commands: `ls` (list files), `cp` (copy), `mv` (move/rename), `rm` (remove files).
- Process commands: `ps` (list processes), `kill` (terminate process).
- Disk commands: `df` (disk space), `du` (disk usage).
- Networking: `ping`, `ifconfig`, `netstat`.

Role:

- Allow users and administrators to interact with the system.
- Often used in shell scripts to automate tasks.

4. Shell

What is the Shell?

The shell is the command interpreter, the user interface for the Linux OS. It accepts commands typed by the user and interprets them to perform actions.

Types of Shells:

- **Bash (Bourne Again Shell):** The most common shell in Linux.
- **Zsh, Tcsh, Ksh, Fish:** Other popular shells with advanced features.

Click Here join official channel

Shell Functions:

- Read user commands.
- Parse and execute commands.
- Supports scripting to automate sequences of commands.
- Provides environment variables and control over input/output.

5. User Applications

What are User Applications?

User applications are software programs running on Linux, providing various functionalities from simple text editors to complex graphical interfaces.

Categories:

- **Graphical User Interface (GUI) Applications:** Web browsers, office suites, media players.
- **Command-line tools:** Editors like `vim`, file managers, network tools.
- **Daemons:** Background services like web servers (`apache`), database servers (`mysql`).

What are Shell Commands in Linux?

- A shell in Linux is a program that serves as an interface between the user and the operating system. It accepts commands from the user, interprets them, and passes them to the operating system for execution. The commands can be used for a wide range of tasks, from file manipulation to system management.
- Some of the essential basic shell commands in Linux for different operations are:
- File Management -> cp, mv, rm, mkdir
- Navigation -> cd, pwd, ls
- Text Processing -> cat, grep, sort, head
- System Monitoring -> top, ps, df
- Permissions and Ownership -> chmod, chown, chgrp
- Networking - > ping, wget, curl, ssh, scp, ftp
- Compression and Archiving - > tar, gzip, gunzip, zip, unzip
- Package Management - > dnf, yum, apt-get
- Process Management -> kill, killall, bg, killall, **kill**

Click Here join official channel

Basic Shell Commands for File and Directory Management

Command	Description	Example
ls	Lists files and directories	ls
cd	Changes the current directory	cd /home/user/Documents
pwd	Displays the current directory path	pwd
mkdir	Creates a new directory	mkdir new_directory
rm	Removes files or directories	rm file.txt
cp	Copies files or directories	cp file1.txt file2.txt
mv	Moves or renames files and directories	mv old_name new_name
touch	Creates an empty file or updates file timestamps	touch newfile.txt

Examples:

1. List files in a directory:

```
ls
```

2. Change directory:

```
cd/home/user
```

3. Create a new directory:

```
mkdir new_directory
```

4. Copy a file from one location to another:

```
cp source.txt destination.txt
```

5. Remove a file:

```
rm file.txt
```

Text Processing Commands in Linux

Command	Description	Example
cat	Displays the contents of a file	cat file.txt
grep	Searches for a pattern in a file	grep "error" log.txt
sort	Sorts the contents of a file	sort file.txt
head	Displays the first few lines of a file	head file.txt

Click Here join official channel

Command	Description	Example
`tail`	Displays the last few lines of a file	`tail file.txt`
`wc`	Counts the lines, words, and characters in a file	`wc file.txt`

Examples:

1. Display the contents of a file:

```
cat file.txt
```

2. Search for a pattern in a file:

```
grep "error" file.txt
```

3. Sort the contents of a file:

```
sort file.txt
```

4. Display the first 10 lines of a file:

```
head file.txt
```

5. Display the last 10 lines of a file:

```
tail file.txt
```

File Permissions and Ownership Commands

Command	Description	Example
`chmod`	Changes file permissions	`chmod 755 file.txt`
`chown`	Changes file owner and group	`chown user:group file.txt`
`chgrp`	Changes file group ownership	`chgrp group file.txt`

Examples:

1. Change permissions of a file:

```
chmod 755 file.txt
```

2. Change the owner of a file:

```
chown user:group file.txt
```

System Monitoring and Process Management Commands

Command	Description	Example
`top`	Displays real-time system information (CPU, memory)	`top`
`ps`	Displays the list of running processes	`ps aux`
`kill`	Terminates a process by its ID	`kill 1234`
`df`	Displays disk space usage	`df -h`

Click Here join official channel

Examples:

1. View running processes:

```
ps aux
```

2. Display real-time system statistics:

```
top
```

3. Kill a process by its ID:

```
kill 1234
```

4. Check disk space usage:

```
df -h
```

Networking Shell Commands

Command	Description	Example
`ping`	Checks the network connection to a server	`ping example.com`
`wget`	Retrieves files from the web	`wget http://example.com///file.zip`
`curl`	Transfers data from or to a server	`curl http://example.com//`
`ssh`	Opens SSH client (remote login program)	`ssh user@example.com`
`scp`	Securely copies files between hosts	`scp file.txt user@example.com:/path/`
`ftp`	Transfers files using the File Transfer Protocol	`ftp ftp.example.com`

Examples

1. Check the network connection to a server:

- **Command:** ping
- **Example:** ping example.com

2. Retrieve files from the web:

- **Command:** wget
- **Example:** wget http://example.com///file.zip

3. Transfer data from or to a server:

Click Here join official channel

- **Command:** curl
- **Example:** curl http://example.com//

4. **Open SSH client (remote login program):**

- **Command:** ssh
- **Example**: ssh user@example.com

5. Securely copy files between hosts:

- **Command:** scp
- **Example:** scp file.txt user@example.com:/path/

6. Transfer files using the File Transfer Protocol:

- **Command:** ftp
- **Example:** ftp ftp.example.com

Advanced Shell Commands

Command	Description	Example
`find`	Searches for files and directories	`find /home/user -name "*.txt"`
`tar`	Archives files into a tarball (.tar) or extracts them	`tar -cvf archive.tar file1.txt file2.txt`
`ssh`	Connects to a remote machine via SSH	`ssh user@remote_host`

Examples:

1. Find files in a directory:

```
find /home/user -name "*.txt"
```

2. Create a tarball archive:

```
tar -cvf archive.tar file1.txt file2.txt
```

3. Connect to a remote machine using SSH:

```
ssh user@remote_host
```

- **Using Shell Command Piping**

You can combine **multiple commands** by piping their output. In short, it allows the **output of one command** to be used as the **input for another command**.

Examples:

1. View the top 10 processes:

```
ps aux | head -n 10
```

2. Search and sort a log file:

```
grep "error" log.txt | sort
```



CLICK HERE TO JOIN WHATSAPP

CHANNEL FOR SPPU UNIVERSITY

JUST CLICK ON LINK

https://www.whatsapp.com/channel/0029ValjFrilCVfpcV9HFc3b